Name:- Mali Akash **ROLL NO:** COB53

Div: B **Batch : B3**

Practical No. 01

Data Wrangling Perform the following operations using Python on any open source dataset (e.g., data.csv)

1. Import all the required Python Libraries.
2. Locate an open source data from the web (e.g. https://www.kaggle.com). Provide a clear description of the data and its source (i.e., URL of the web site).
3. Load the Dataset into pandas data frame.
4. Data Preprocessing: check for missing values in the data using pandas insult(), describe() function to get some initial statistics. Provide variable descriptions. Types of variables etc. Check the dimensions of the data frame.
5. Data Formatting and Data Normalization: Summarize the types of variables by checking the data types (i.e., character, numeric, integer, factor, and logical) of the variables in the data set. If variables are not in the correct data type, apply proper type conversions.
6. Turn categorical variables into quantitative variables in Python. In addition to the codes and outputs, explain every operation that you do in the above steps and explain everything that you do to import/read/scrape the data set.

CODE: import pandas as

pd import

numpy as np

df = pd.read_csv('StudentsPerformance.csv') df.head()

OUTPUT:

[2]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

CODE:

df.isnull().sum() **OUTPUT:**

```
[3]: gender                         0
     race/ethnicity                 0
     parental level of education    0
     lunch                          0
     test preparation course        0
     math score                     0
     reading score                  0
     writing score                  0
     dtype: int64
```

CODE: df.describe()

OUTPUT:

	math score	reading score	writing score
count	1000.00000	1000.000000	1000.000000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

CODE: df.notnull().sum()

OUTPUT:

```
gender                         1000
race/ethnicity                 1000
parental level of education    1000
lunch                          1000
test preparation course        1000
math score                     1000
reading score                  1000
writing score                  1000
dtype: int64
```

CODE:

df.size

OUTPUT: 8000

CODE: df.ndim

OUTPUT:

2

CODE: df.shape

OUTPUT:

(1000, 8) **CODE:**
df.info()

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   gender                       1000 non-null   object
 1   race/ethnicity               1000 non-null   object
 2   parental level of education  1000 non-null   object
 3   lunch                        1000 non-null   object
 4   test preparation course      1000 non-null   object
 5   math score                   1000 non-null   int64
 6   reading score                1000 non-null   int64
 7   writing score                1000 non-null   int64
dtypes: int64(3), object(5)
memory usage: 62.6+ KB
```

CODE: df['writing score'].astype(int) **OUTPUT:**

```
0      74
1      88
2      93
3      44
4      75
      ..
995    95
996    55
997    65
998    77
999    86
Name: writing score, Length: 1000, dtype: int64
```

CODE: df.dropna()

df.isnull().sum() **OUTPUT:**

```
gender                         0
race/ethnicity                 0
parental level of education    0
lunch                          0
test preparation course        0
math score                     0
reading score                  0
writing score                  0
dtype: int64
```

CODE:

df['writing score'] = df['writing score'].astype(float)

df.head()

OUTPUT:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74.0
1	female	group C	some college	standard	completed	69	90	88.0
2	female	group B	master's degree	standard	none	90	95	93.0
3	male	group A	associate's degree	free/reduced	none	47	57	44.0
4	male	group C	some college	standard	none	76	78	75.0

CODE: df["gender"] =

df['gender'].replace({"female":0,"male":1})

df.sample(5) **OUTPUT:**

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
479	1	group E	associate's degree	standard	none	76	71	67
813	1	group E	some high school	standard	completed	87	84	76
420	0	group C	associate's degree	free/reduced	completed	82	93	93
296	1	group A	some high school	standard	completed	46	41	43
245	1	group C	associate's degree	standard	none	85	76	71

Name:- Mali Akash **ROLL NO: COB53**

Div: B **Batch : B3**

Practical No. 02

Data Wrangling II Create an "Academic performance" dataset of students and perform the following operations using Python.

1. Scan all variables for missing values and inconsistencies. If there are missing values and/or inconsistencies, use any of the suitable techniques to deal with them.
2. Scan all numeric variables for outliers. If there are outliers, use any of the suitable techniques to deal with them.
3. Apply data transformations on at least one of the variables. The purpose of this transformation should be one of the following reasons: to change the scale for better understanding of the variable, to convert a non-linear relation into a linear one, or to decrease the skewness and convert the distribution into a normal distribution. Reason and document your approach properly.

CODE:

```
import pandas as pd import

matplotlib.pyplot as plt import

numpy as np np.version.version
```

OUTPUT:

```
'2.1.3'
```

CODE:

```
df = pd.read_csv("StudentsPerformance.csv") df.head()
```

OUTPUT:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

CODE:

```
df.isnull().sum()
```

OUTPUT:

```
gender                         0
race/ethnicity                 0
parental level of education    0
lunch                          0
test preparation course        0
math score                     0
reading score                  0
writing score                  0
dtype: int64
```

CODE:

df.dropna()

OUTPUT:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75
...	...	...	...	...	...	...	...	...
995	female	group E	master's degree	standard	completed	88	99	95
996	male	group C	high school	free/reduced	none	62	55	55
997	female	group C	high school	free/reduced	completed	59	71	65
998	female	group D	some college	standard	completed	68	78	77
999	female	group D	some college	free/reduced	none	77	86	86

1000 rows × 8 columns

CODE:

math_score_mean = df["math score"].mean() df["math

score"] = df["math score"].fillna(math_score_mean) df.head()

OUTPUT:

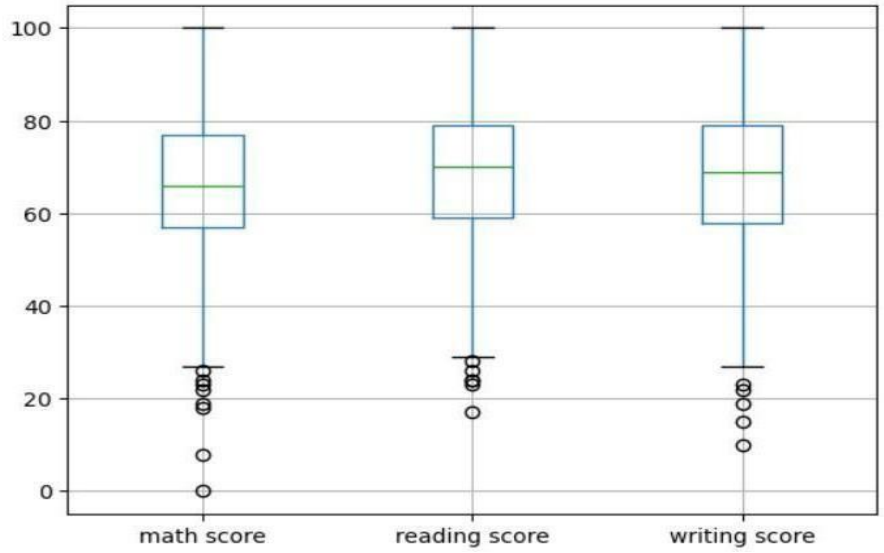
	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

CODE:

import matplotlib as plt df.boxplot()

OUTPUT:

<Axes: >



CODE:

newdf = df[df["math score"] > 30] newdf.head()

OUTPUT:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

CODE: newdf.boxplot()

OUTPUT: <Axes: >

CODE: pip uninstall numpy

CODE: Y

CODE: pip install numpy==2.0

Name: Mali Akash **ROLL NO: COB53**

Div: B **Batch : B3**

Practical No. 03

Descriptive Statistics - Measures of Central Tendency and variability Perform the following operations on any open source dataset (e.g., data.csv)

1. Provide summary statistics (mean, median, minimum, maximum, standard deviation) for a dataset (age, income etc.) with numeric variables grouped by one of the qualitative (categorical) variable. For example, if your categorical variable is age groups and quantitative variable is income, then provide summary statistics of income grouped by the age groups. Create a list that contains a numeric value for each response to the categorical variable.
2. Write a Python program to display some basic statistical details like percentile, mean, standard deviation etc. of the species of 'Iris-setosa', 'Iris-versicolor' and 'Iris- versicolor' of iris.csv dataset. Provide the codes with outputs and explain everything that you do in this step.

CODE:

```
import pandas as pd df =

pd.read_csv("Iris.csv") df.head()
```

OUTPUT:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

CODE: df.describe()

OUTPUT:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

CODE:

df["SepalLengthCm"].describe() **OUTPUT**:

```
:  count    150.000000
   mean       5.843333
   std        0.828066
   min        4.300000
   25%        5.100000
   50%        5.800000
   75%        6.400000
   max        7.900000
   Name: SepalLengthCm, dtype: float64
```

CODE: df.groupby("Species").describe() **OUTPUT**:

	Id								SepalLengthCm		...	PetalLengthCm								PetalWidthCm						
	count	mean	std	min	25%	50%	75%	max	count	mean	...	75%	max	count	mean	std	min	25%	50%	75%	max
Species																					
Iris-setosa	50.0	25.5	14.57738	1.0	13.25	25.5	37.75	50.0	50.0	5.006	...	1.575	1.9	50.0	0.244	0.107210	0.1	0.2	0.2	0.3	0.6
Iris-versicolor	50.0	75.5	14.57738	51.0	63.25	75.5	87.75	100.0	50.0	5.936	...	4.600	5.1	50.0	1.326	0.197753	1.0	1.2	1.3	1.5	1.8
Iris-virginica	50.0	125.5	14.57738	101.0	113.25	125.5	137.75	150.0	50.0	6.588	...	5.875	6.9	50.0	2.026	0.274650	1.4	1.8	2.0	2.3	2.5

3 rows × 40 columns

CODE: df.groupby("Species").describe().sum() **OUTPUT**:

```
Id             count    150.000000
               mean     226.500000
               std       43.732139
               min      153.000000
               25%      189.750000
               50%      226.500000
               75%      263.250000
               max      300.000000
SepalLengthCm  count    150.000000
               mean      17.530000
               std        1.504540
               min       14.100000
               25%       16.625000
               50%       17.400000
               75%       18.400000
               max       20.700000
SepalWidthCm   count    150.000000
               mean       9.162000
               std        1.017319
               min        6.500000
               25%        8.450000
               50%        9.200000
               75%        9.850000
               max       11.600000
PetalLengthCm  count    150.000000
               mean      11.276000
               std        1.195317
               min        8.500000
               25%       10.500000
               50%       11.400000
               75%       12.050000
               max       13.900000
PetalWidthCm   count    150.000000
               mean       3.596000
               std        0.579612
               min        2.500000
               25%        3.200000
               50%        3.500000
               75%        4.100000
               max        4.900000
dtype: float64
```

Name: Mali Akash **ROLL NO:COB53**

Div: B **Batch : B3**

Practical No. 04

Data Analytics I Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset (https://www.kaggle.com/c/boston-housing). The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this dataset. The objective is to predict the value of prices of the house using the given features.

CODE:

```
import pandas as pd import numpy as np from
sklearn.model_selection import train_test_split from
sklearn.linear_model import LinearRegression from
sklearn.metrics import mean_squared_error
```

CODE:

```
df = pd.read_csv("BostonHousing.csv") df
= df.dropna() df.head()
```

OUTPUT:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

CODE:

```
df.columns
```

OUTPUT:

```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',  'ptratio',
'b', 'lstat', 'medv'], dtype='object')
```

CODE:

```
x = df[['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax', 'ptratio', 'b', 'lstat']]  x.head()
```

OUTPUT:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33

CODE:

```
y = df['medv']

y.head()
```

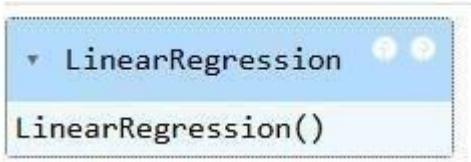
OUTPUT:

```
0    24.0
1    21.6
2    34.7
3    33.4
4    36.2
Name: medv, dtype: float64
```

CODE:

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_state=42) model

= LinearRegression() model.fit(x_train,y_train)
```

OUTPUT:



CODE:

```
y_pred = model.predict(x_test) y_pred
```

OUTUT:

```
array([10.82520289, 22.97716771, 15.45617932, 33.55363131, 22.96357871,
       11.52151263, 12.76018157, 19.74412591, 21.33180568, 11.7372368 ,
       18.75187948, 30.04070255, -0.73011025, 25.78030298,  3.02335542,
        8.49359394, 24.07065874, 18.57018302, 25.24003893, -6.24945751,
       13.33486252, 19.08911255, 27.0053246 , 19.59024598, 22.40273032,
       16.47206196, 28.79995249, 26.24334357, 18.42194929, 21.27338464,
       20.62838908, 30.49181729, 17.87807473, 31.53661897, 31.16125663,
       22.20316674,  7.79878712, 23.70737642,  8.54510946, 25.0261323 ,
       12.99764774, 36.12050346, 14.45054578, 30.51121076, 13.02756177,
       28.48505695, 30.34475695, 20.15771804, 18.46362559, 13.69183882,
       24.00613417, 32.99780499, 16.4544118 , 11.66937979, 34.39689874,
       33.37924364, 17.77929903, 18.70970757, 16.25656178, 27.35347057,
       20.48252629, 40.60322048, 20.53694472,  8.20383246, 25.97767891,
       27.81783878, 12.08008232,  7.62795819, 27.14868012, 16.44871208,
       23.46295285, 14.63324084, 40.28319824, 28.66936219, 23.1422757 ,
       23.95467347, 35.49409707, 24.49032705, 20.75456047, 15.97157605,
       27.18392572, 27.90827964, 21.23340735, 29.37584949, 23.9104647 ,
       29.29067164, 24.22591482, 20.08729338, 18.20901184, 44.2614741 ,
        4.63790216, 19.31301769, 17.2763475 , 23.72223401,  7.38111706,
       17.02032604, 31.01206401, 21.14872276, 10.9653362 , 20.85193641,
       24.18859543, 17.31441353, 12.19815419, 19.11197493, 19.50296116,
       22.01189876, 35.62919117, 31.55632051, 20.24630891, 20.2365227 ,
       14.26461161, 11.71171865, 21.66497318, 15.74320729, 20.29084409,
       19.52326714, 25.23052357, 23.83851879, 23.14474265, 22.78985166,
       40.21608485, 27.45423907, 24.8064738 , 30.06864408, 30.07124307,
       38.69282771])
```

CODE: model.score(x_train,y_train)

OUTPUT: 0.7335900413194543

CODE: model.score(x_test,y_test)

OUTPUT: 0.7459403901980342

CODE: np.sqrt(mean_squared_error(y_test,y_pred))

OUTPUT: 4.387285229095361

Name: Mali Akash **ROLL NO: COB53**

Div: B **Batch : B3**

Practical No. 05

Data Analytics II

1. Implement logistic regression using Python/R to perform classification on Social_Network_Ads.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
```

CODE:

```
df = pd.read_csv("Social_Network_Ads.csv")
df['Gender'] = df['Gender'].replace({"Male":0,"Female":1})
df.head()
```

OUTPUT:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	0	19	19000	0
1	15810944	0	35	20000	0
2	15668575	1	26	43000	0
3	15603246	1	27	57000	0
4	15804002	0	19	76000	0

CODE:

```
df.columns
```

OUTPUT:

```
Index(['User ID', 'Gender', 'Age', 'EstimatedSalary', 'Purchased'], dtype='object')
```

CODE:

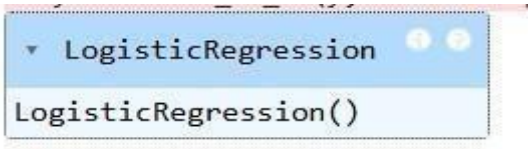
```
x = df[['User ID', 'Gender', 'Age', 'EstimatedSalary']]
y = df[['Purchased']]
```

CODE: x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_state=29)

model = LogisticRegression()

model.fit (x_train,y_train)

OUTPUT:



CODE: y_pred = model.predict(x_test)

y_pred

OUTPUT:

```
array([0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0,
       0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1])
```

CODE: model.score(x_train,y_train)

OUTPUT: 0.8633333333333333

CODE: model.score(x,y)

OUTPUT: 0.85

CODE: cm = confusion_matrix(y_test,y_pred)

cm

OUTPUT: array([[63, 6], [13, 18]])

CODE: tn, fp, fn, tp = confusion_matrix(y_test,y_pred).ravel()

CODE: print (tn, fp, fn, tp)

OUTPUT: 63 6 13 18

CODE: a = accuracy_score(y_test,y_pred)

a

OUTPUT:0.81

CODE: e = 1 - a

e

OUTPUT: 0.18999999999999995

CODE: precision_score(y_test,y_pred)

OUTPUT: 0.75

CODE: recall_score(y_test,y_pred)

OUTPUT: 0.5806451612903226

Name: Mali Akash **ROLL NO: COB53**

Div: B **Batch : B3**

Practical No. 06

Data Analytics III

1. Implement Simple Naïve Bayes classification algorithm using Python/R on iris.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

CODE: import pandas as pd import numpy as np from sklearn.preprocessing import MinMaxScaler from sklearn.model_selection import train_test_split from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score from sklearn.naive_bayes import GaussianNB

df = pd.read_csv("Iris.csv") df.head() **OUTPUT**:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

CODE:

x = df.drop(['Species'],axis = 1) y = df['Species']

scaler = MinMaxScaler() x_scaled = scaler.fit_transform(x) x_scaled

OUTPUT:

```
array([[0.        , 0.22222222, 0.625     , 0.06779661, 0.04166667],
       [0.00671141, 0.16666667, 0.41666667, 0.06779661, 0.04166667],
       [0.01342282, 0.11111111, 0.5       , 0.05084746, 0.04166667],
       [0.02013423, 0.08333333, 0.45833333, 0.08474576, 0.04166667],
       [0.02684564, 0.19444444, 0.66666667, 0.06779661, 0.04166667],
       [0.03355705, 0.30555556, 0.79166667, 0.11864407, 0.125     ],
       [0.04026846, 0.08333333, 0.58333333, 0.06779661, 0.08333333],
       [0.04697987, 0.19444444, 0.58333333, 0.08474576, 0.04166667],
       [0.05369128, 0.02777778, 0.375     , 0.06779661, 0.04166667],
       [0.06040268, 0.16666667, 0.45833333, 0.08474576, 0.        ],
       [0.06711409, 0.30555556, 0.70833333, 0.08474576, 0.04166667],
       [0.0738255 , 0.13888889, 0.58333333, 0.10169492, 0.04166667],
       [0.08053691, 0.13888889, 0.41666667, 0.06779661, 0.        ],
       [0.08724832, 0.        , 0.41666667, 0.01694915, 0.        ],
       [0.09395973, 0.41666667, 0.83333333, 0.03389831, 0.04166667],
       [0.10067114, 0.38888889, 1.        , 0.08474576, 0.125     ],
       [0.10738255, 0.30555556, 0.79166667, 0.05084746, 0.125     ],
       [0.11409396, 0.22222222, 0.625     , 0.06779661, 0.08333333],
       [0.12080537, 0.38888889, 0.75      , 0.11864407, 0.08333333],
       [0.12751678, 0.22222222, 0.75      , 0.08474576, 0.08333333],
       [0.13422819, 0.30555556, 0.58333333, 0.11864407, 0.04166667],
       [0.1409396 , 0.22222222, 0.70833333, 0.08474576, 0.125     ],
       [0.14765101, 0.08333333, 0.66666667, 0.        , 0.04166667],
       [0.15436242, 0.22222222, 0.54166667, 0.11864407, 0.16666667],
       [0.16107383, 0.13888889, 0.58333333, 0.15254237, 0.04166667],
       [0.16778523, 0.19444444, 0.41666667, 0.10169492, 0.04166667],
       [0.17449664, 0.19444444, 0.58333333, 0.10169492, 0.125     ],
       [0.18120805, 0.25      , 0.625     , 0.08474576, 0.04166667],
       [0.18791946, 0.25      , 0.58333333, 0.06779661, 0.04166667],
       [0.19463087, 0.11111111, 0.5       , 0.10169492, 0.04166667],
       [0.20134228, 0.13888889, 0.45833333, 0.10169492, 0.04166667],
       [0.20805369, 0.30555556, 0.58333333, 0.08474576, 0.125     ],
       [0.2147651 , 0.25      , 0.875     , 0.08474576, 0.        ],
       [0.22147651, 0.33333333, 0.91666667, 0.06779661, 0.04166667],
       [0.22818792, 0.16666667, 0.45833333, 0.08474576, 0.        ],
       [0.23489933, 0.19444444, 0.5       , 0.03389831, 0.04166667],
       [0.24161074, 0.33333333, 0.625     , 0.05084746, 0.04166667],
       [0.24832215, 0.16666667, 0.45833333, 0.08474576, 0.        ],
       [0.25503356, 0.02777778, 0.41666667, 0.05084746, 0.04166667],
```

```
       [0.73154362, 0.80555556, 0.66666667, 0.86440678, 1.        ],
       [0.73825503, 0.61111111, 0.5       , 0.69491525, 0.79166667],
       [0.74496644, 0.58333333, 0.29166667, 0.72881356, 0.75      ],
       [0.75167785, 0.69444444, 0.41666667, 0.76271186, 0.83333333],
       [0.75838926, 0.38888889, 0.20833333, 0.6779661 , 0.79166667],
       [0.76510067, 0.41666667, 0.33333333, 0.69491525, 0.95833333],
       [0.77181208, 0.58333333, 0.5       , 0.72881356, 0.91666667],
       [0.77852349, 0.61111111, 0.41666667, 0.76271186, 0.70833333],
       [0.7852349 , 0.94444444, 0.75      , 0.96610169, 0.875     ],
       [0.79194631, 0.94444444, 0.25      , 1.        , 0.91666667],
       [0.79865772, 0.47222222, 0.08333333, 0.6779661 , 0.58333333],
       [0.80536913, 0.72222222, 0.5       , 0.79661017, 0.91666667],
       [0.81208054, 0.36111111, 0.33333333, 0.66101695, 0.79166667],
       [0.81879195, 0.94444444, 0.33333333, 0.96610169, 0.79166667],
       [0.82550336, 0.55555556, 0.29166667, 0.66101695, 0.70833333],
       [0.83221477, 0.66666667, 0.54166667, 0.79661017, 0.83333333],
       [0.83892617, 0.80555556, 0.5       , 0.84745763, 0.70833333],
       [0.84563758, 0.52777778, 0.33333333, 0.6440678 , 0.70833333],
       [0.85234899, 0.5       , 0.41666667, 0.66101695, 0.70833333],
       [0.8590604 , 0.58333333, 0.33333333, 0.77966102, 0.83333333],
       [0.86577181, 0.80555556, 0.41666667, 0.81355932, 0.625     ],
       [0.87248322, 0.86111111, 0.33333333, 0.86440678, 0.75      ],
       [0.87919463, 1.        , 0.75      , 0.91525424, 0.79166667],
       [0.88590604, 0.58333333, 0.33333333, 0.77966102, 0.875     ],
       [0.89261745, 0.55555556, 0.33333333, 0.69491525, 0.58333333],
       [0.89932886, 0.5       , 0.25      , 0.77966102, 0.54166667],
       [0.90604027, 0.94444444, 0.41666667, 0.86440678, 0.91666667],
       [0.91275168, 0.55555556, 0.58333333, 0.77966102, 0.95833333],
       [0.91946309, 0.58333333, 0.45833333, 0.76271186, 0.70833333],
       [0.9261745 , 0.47222222, 0.41666667, 0.6440678 , 0.70833333],
       [0.93288591, 0.72222222, 0.45833333, 0.74576271, 0.83333333],
       [0.93959732, 0.66666667, 0.45833333, 0.77966102, 0.95833333],
       [0.94630872, 0.72222222, 0.45833333, 0.69491525, 0.91666667],
       [0.95302013, 0.41666667, 0.29166667, 0.69491525, 0.75      ],
       [0.95973154, 0.69444444, 0.5       , 0.83050847, 0.91666667],
       [0.96644295, 0.66666667, 0.54166667, 0.79661017, 1.        ],
       [0.97315436, 0.66666667, 0.41666667, 0.71186441, 0.91666667],
       [0.97986577, 0.55555556, 0.20833333, 0.6779661 , 0.75      ],
       [0.98657718, 0.61111111, 0.41666667, 0.71186441, 0.79166667],
       [0.99328859, 0.52777778, 0.58333333, 0.74576271, 0.91666667],
       [1.        , 0.44444444, 0.41666667, 0.69491525, 0.70833333]])
```

CODE: x_train,x_test,y_train,y_test = train_test_split(x_scaled,y,test_size = 0.2, random_state = 43)

gnb = GaussianNB() gnb.fit(x_train,y_train)

y_pred

= gnb.predict(x_test)

cm = confusion_matrix(y_test,y_pred) cm

OUTPUT: array([[13, 0, 0],

[0, 8, 0],

[0, 0, 9]])

CODE:tn = confusion_matrix(y_test,y_pred).ravel() fp

= confusion_matrix(y_test,y_pred).ravel() fn=

confusion_matrix(y_test,y_pred).ravel() tp =

confusion_matrix(y_test,y_pred).ravel() print (tn,

fp, fn, tp)

OUTPUT: [13 0 0 0 8 0 0 0 9] [13 0 0 0 8 0 0 0 9] [13 0 0 0 8 0 0 0 9] [13 0 0 0 8 0 0 0 9]

CODE: a = accuracy_score(y_test,y_pred) a

OUTPUT:1.0

CODE: e = 1 - a e

OUTPUT:0.0

CODE: precision_score(y_test,y_pred,average='micro')

OUTPUT:1.0

CODE: recall_score(y_test,y_pred,average='micro')

OUTPUT:1.0

Name:- Mali Akash **ROLL NO: COB53**

Div: B **Batch : B3**

Practical No. 07

Text Analytics

1. Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization.

2. Create representation of document by calculating Term Frequency and Inverse Document Frequency.

CODE: !pip install nltk

import nltk import nltk

nltk.download('punkt')

nltk.download('punkt_tab')

import re nltk.download('punkt')

nltk.download('stopwords')

nltk.download('wordnet')

nltk.download('averaged_perceptron_tagger')

nltk.download('omw-1.4') **OUTPUT**:

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\prana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\prana\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\prana\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\prana\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\prana\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\prana\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     C:\Users\prana\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
True
```

CODE:

text = "Tokenization is the first step in text analytics."

from nltk.tokenize import sent_tokenize tokenized_text

= sent_tokenize(text) print(tokenized_text)

OUTPUT: ['Tokenization is the first step in text analytics.'] **CODE**:

from nltk.tokenize import word_tokenize tokenized_word

= word_tokenize(text) print(tokenized_word)

OUTPUT: ['Tokenization', 'is', 'the', 'first', 'step', 'in', 'text', 'analytics', '.']

CODE: from nltk.corpus import stopwords stop_words =

set(stopwords.words("english")) print(stop_words) **OUTPUT**:

```
{'our', 'isn', "hasn't", 'until', 'itself', 'once', 'their', "we've", 'y', 'through', 're', 'for', 'ca
n', "she's", "they're", 'his', 'whom', "we're", 'at', 'myself', 'these', 'themselves', 'have', "you've
e", 'o', 'me', 'more', "should've", "they'd", 'she', 'under', 'are', "don't", 'yourself', 'wouldn',
'm', 'shan', 'couldn', 'ours', 'don', 'down', 't', 'out', "needn't", "he'd", "we'd", "mightn't", 'the
y', 'was', 'what', 'shouldn', "couldn't", 'but', 'why', 'were', 'about', 'a', 'i', 'such', 'or', "tha
t'll", 'her', 'doing', 'and', 'than', 'that', "you're", 'yours', 'doesn', 'to', 'who', 've', "doesn't",
'while', 'now', 'herself', 'very', 's', 'd', 'the', 'between', 'my', 'where', 'himself', 'we', 'be', 'j
ust', 'you', 'with', "hadn't", 'again', 'theirs', 'as', 'only', 'hasn', "i'm", 'aren', 'needn', 'afte
r', 'by', "you'd", 'other', 'some', 'because', 'not', "won't", 'he', 'before', 'there', 'here', "they've
e", 'ain', "haven't", 'when', 'in', 'it', "mustn't", 'no', 'mustn', 'same', "weren't", 'which', 'll',
"she'd", 'too', 'any', 'didn', 'him', 'mightn', "i'll", 'its', "aren't", 'on', "shan't", "we'll", 'abov
e', "it's", 'own', 'hadn', "isn't", 'ma', 'them', 'during', 'been', 'an', "i'd", 'off', 'into', 'your',
"it'd", 'both', 'each', 'few', 'how', 'nor', 'will', 'up', 'won', 'hers', "it'll", "he'll", "i've", 'al
l', 'of', 'has', 'so', "she'll", 'this', 'those', "wouldn't", "didn't", 'should', 'do', 'does', 'oursel
ves', 'haven', 'below', "you'll", 'having', 'being', 'is', 'then', 'against', 'if', 'weren', 'had', 'di
d', 'yourselves', 'over', 'most', 'wasn', "wasn't", "he's", "they'll", 'further', "shouldn't", 'am', 'f
rom'}
```

CODE: text = "How to remove stop words with NLTK library in

Python?" text = re.sub('[^a-zA-Z]',' ',text) tokens =

word_tokenize(text.lower()) filtered_text = [] for w in tokens:

if w not in stop_words:

filtered_text.append(w) print("Tokenized Sentence

:",tokens) print("Filtered Sentence :",filtered_text)

OUTPUT:

Tokenized Sentence : ['how', 'to', 'remove', 'stop', 'words', 'with', 'nltk', 'library', 'in', 'python']

Filtered Sentence : ['remove', 'stop', 'words', 'nltk', 'library', 'python'] **CODE**: from nltk.stem import

PorterStemmer e_words = ["wait", "waiting", "waited", "waits"] ps =

PorterStemmer() for w in e_words: rootWord = ps.stem(w) print(rootWord)

OUTPUT: wait **CODE**: from nltk.stem import WordNetLemmatizer import nltk

nltk.download('averaged_perceptron_tagger_eng')

wordnet_lemmatizer = WordNetLemmatizer() text = "studies studying cries cry"

tokenization = nltk.word_tokenize(text) for w in tokenization: print("Lemma for {}

is {}".format(w, wordnet_lemmatizer.lemmatize(w))) **OUTPUT**:

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\prana\AppData\Roaming\nltk_data...
Lemma for studies is study
Lemma for studying is studying
Lemma for cries is cry
Lemma for cry is cry
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger_eng.zip.
```

CODE:

```
from nltk.tokenize import word_tokenize data
= "The pink sweater fit her perfectly" words =
word_tokenize(data) for word in words:
print(nltk.pos_tag([word]))
```

OUTPUT:

```
[('The', 'DT')]
[('pink', 'NN')]
[('sweater', 'NN')]
[('fit', 'NN')]
[('her', 'PRP$')] [('perfectly',
'RB')]
```

CODE:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
d0 = 'Jupiter is the largest Planet' d1 = 'Mars is the fourth
planet from the sun' string = [d0,d1] tfidf = TfidfVectorizer()
result = tfidf.fit_transform(string) print('Word indices:',
tfidf.vocabulary_) print('TF-IDF
Values:', result)
```

OUTPUT:

```
Word indices: {'jupiter': 3, 'is': 2, 'the': 8, 'largest': 4, 'planet': 6, 'mars': 5, 'fourth': 0, 'fro
m': 1, 'sun': 7}
TF-IDF Values: <Compressed Sparse Row sparse matrix of dtype 'float64'
        with 12 stored elements and shape (2, 9)>
  Coords        Values
  (0, 3)        0.5330978245262535
  (0, 2)        0.3793034928087496
  (0, 8)        0.3793034928087496
  (0, 4)        0.5330978245262535
  (0, 6)        0.3793034928087496
  (1, 2)        0.2682080718928097
  (1, 8)        0.5364161437856194
  (1, 6)        0.2682080718928097
  (1, 5)        0.37695708675831013
  (1, 0)        0.37695708675831013
  (1, 1)        0.37695708675831013
  (1, 7)        0.37695708675831013
```

Name: Mali Akash **ROLL NO: COB53**

Div: B **Batch : B3**

Practical No. 08

Data Visualization I

1. Use the inbuilt dataset 'titanic'. The dataset contains 891 rows and contains information about the passengers who boarded the unfortunate Titanic ship. Use the Seaborn library to see if we can find any patterns in the data.
2. Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.

CODE:

```
import numpy as np import
pandas as pd import seaborn as
sb import matplotlib.pyplot as
plt
```

df = sb.load_dataset('titanic') df.head() **OUTPUT**:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

CODE: df.info() **OUTPUT**:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

CODE: print("Missing Values: ") print(df.isnull().sum()) **OUTPUT**:

```
Missing Values:
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

CODE: df['age'].fillna(df['age'].median(), inplace=True)

df.isnull().sum() **OUTPUT**:

```
survived         0
pclass           0
sex              0
age              0
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

CODE:

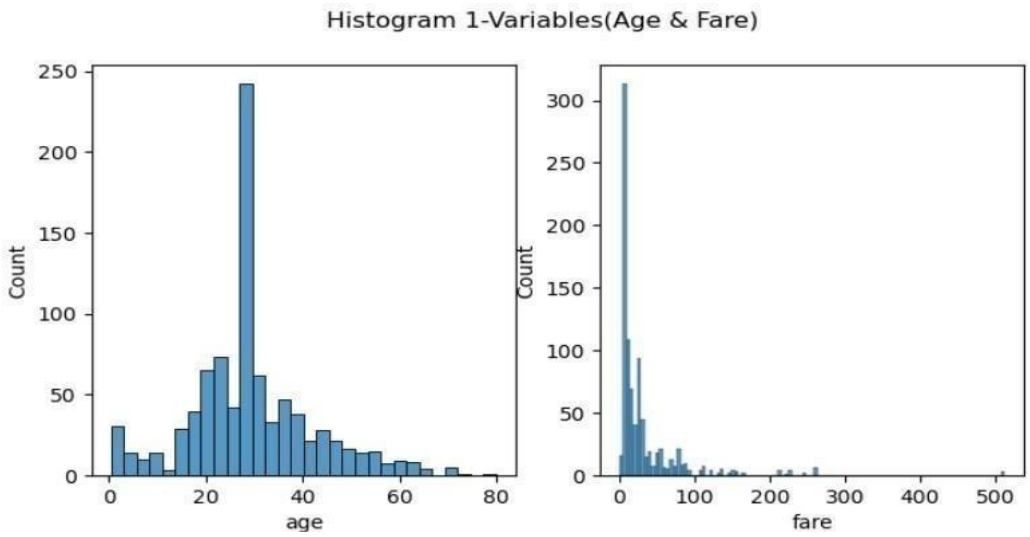
fig, axes = plt.subplots(1,2, figsize=(8,4))

fig.suptitle('Histogram 1-Variables(Age & Fare)')

sb.histplot(data = df, x = 'age', ax = axes[0]) sb.histplot(data

= df, x = 'fare', ax = axes[1]) plt.show()

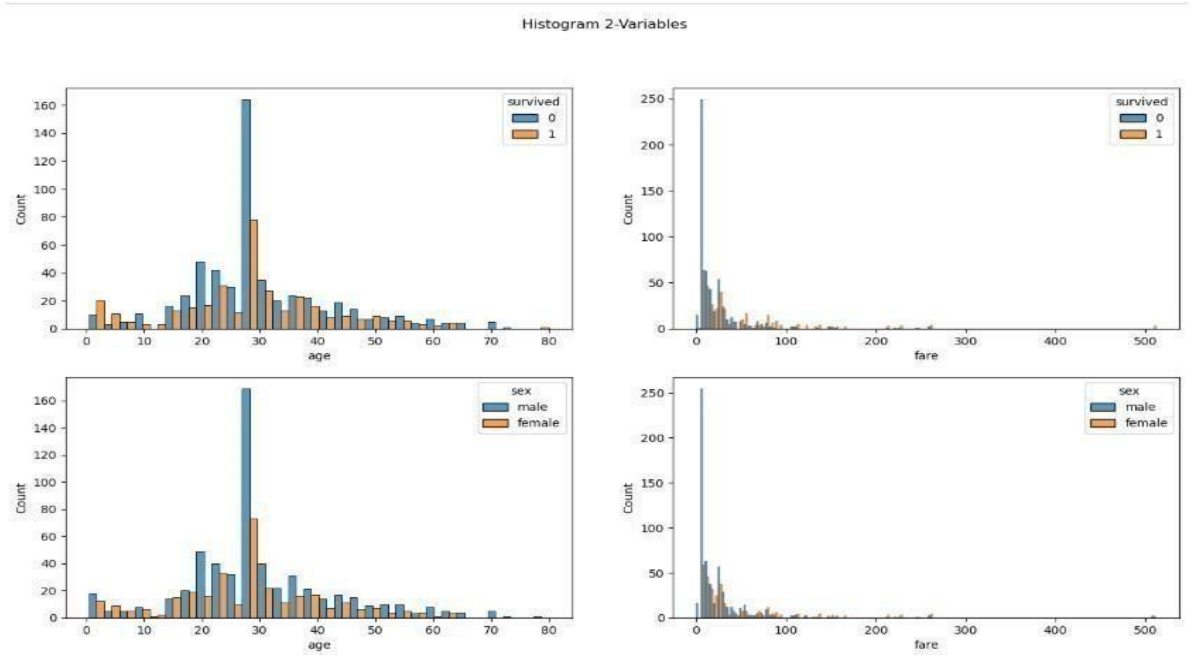
OUTPUT:



CODE:

```
fig, axes = plt.subplots(2,2, figsize = (16,9)) fig.suptitle('Histogram
2-Variables')  sb.histplot(data = df, x = 'age', hue = 'survived', multiple='dodge',
ax = axes[0][0])
sb.histplot(data = df, x = 'fare', hue = 'survived', multiple='dodge', ax = axes[0][1])
sb.histplot(data = df, x = 'age', hue = 'sex', multiple='dodge', ax = axes[1][0])
sb.histplot(data = df, x = 'fare', hue = 'sex', multiple='dodge', ax = axes[1][1]) plt.show()
```

OUTPUT:



Name: Mali Akash **ROLL NO: COB53**

Div: B **Batch : B3**

Practical No. 09

Data Visualization II

1. Use the inbuilt dataset 'titanic' as used in the above problem. Plot a box plot for distribution of age with respect to each gender along with the information about whether they survived or not. (Column names : 'sex' and 'age')
2. Write observations on the inference from the above statistics.

CODE:

```
import seaborn as sns

df = sns.load_dataset('titanic')

df.head()
```

OUTPUT:

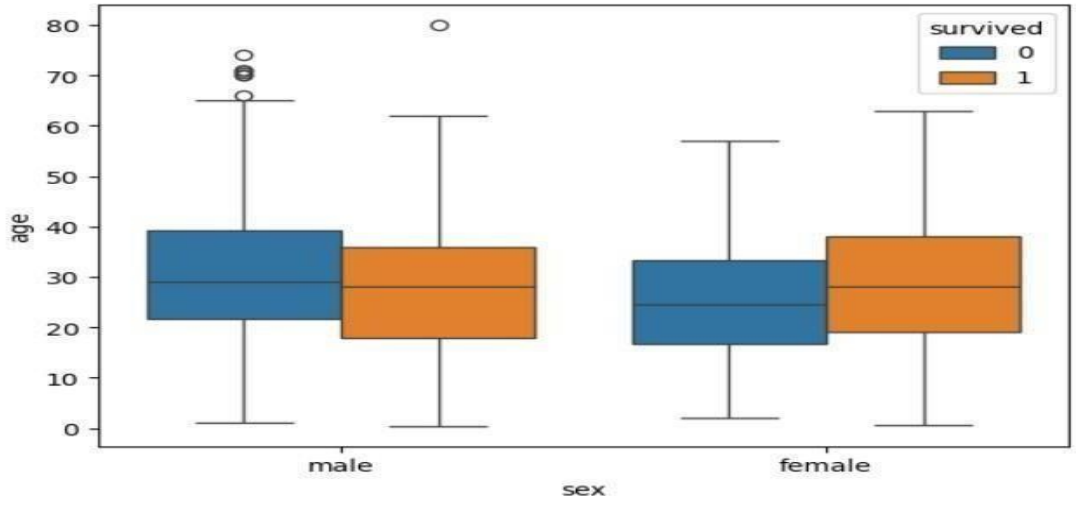
	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

CODE:

```
sns.boxplot(x='sex',y='age',data=df,  hue='survived')
```

OUTPUT:

```
<Axes: xlabel='sex', ylabel='age'>
```



Name: Mali Akash **ROLL NO: COB53**

Div: B **Batch: B2**

Practical No:10

Data Visualization III Download the Iris flower dataset or any other dataset into a DataFrame. (e.g., https://archive.ics.uci.edu/ml/datasets/Iris). Scan the dataset and give the inference as:

1. List down the features and their types (e.g., numeric, nominal) available in the dataset.
2. Create a histogram for each feature in the dataset to illustrate the feature distributions.
3. Create a box plot for each feature in the dataset.
4. Compare distributions and identify outliers.

CODE:

```
import seaborn as sb
ds = sb.load_dataset('iris')
ds.head()
```

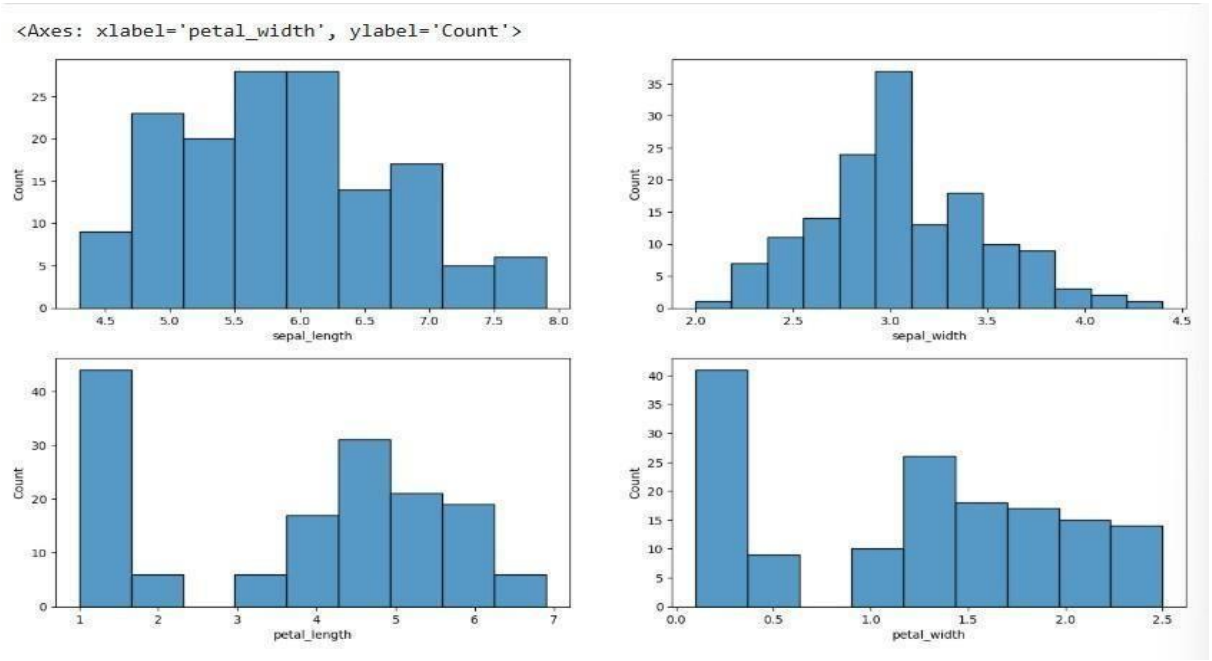
OUTPUT:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

CODE:

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2,2,figsize = (16,9))
sb.histplot(ds['sepal_length'], ax = axes[0,0])
sb.histplot(ds['sepal_width'], ax = axes[0,1])
sb.histplot(ds['petal_length'], ax = axes[1,0])
sb.histplot(ds['petal_width'], ax = axes[1,1])
```

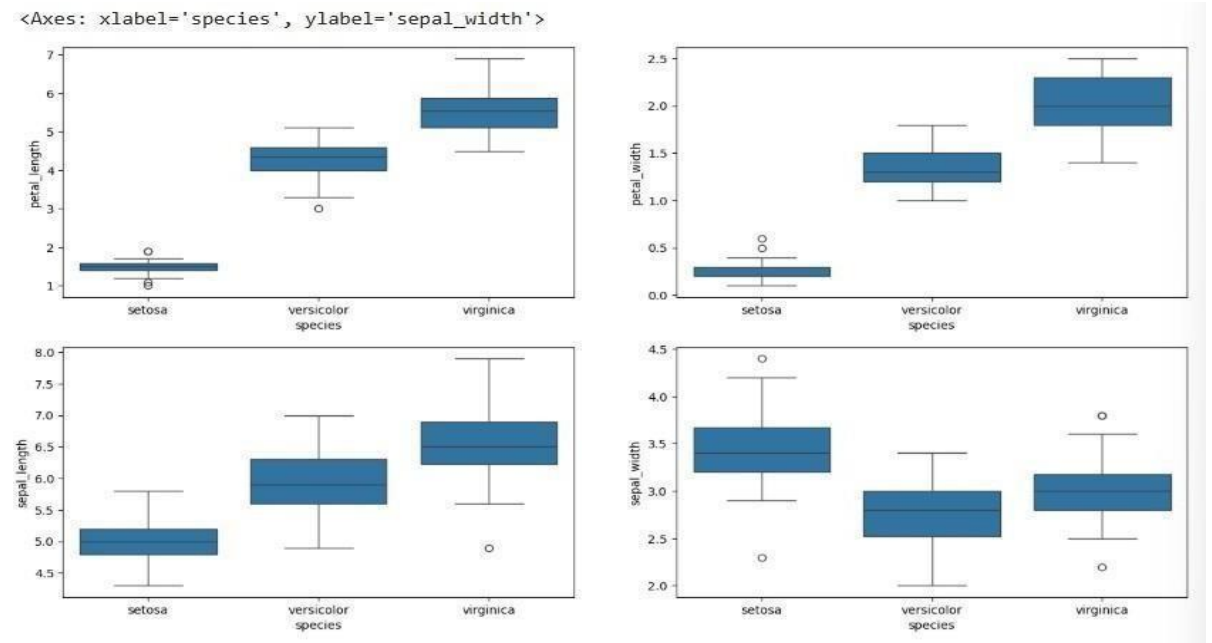
OUTPUT:



CODE:

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2,2,figsize=(16,9))
sb.boxplot(x='species',y='petal_length',data = ds, ax=axes[0,0])
sb.boxplot(x='species',y='petal_width',data = ds, ax=axes[0,1])
sb.boxplot(x='species',y='sepal_length',data = ds, ax=axes[1,0])
sb.boxplot(x='species',y='sepal_width',data = ds, ax=axes[1,1])
```

OUTPUT



Name : Mali Akash

Roll No: COB53

Batch : B3

Div : B

.....

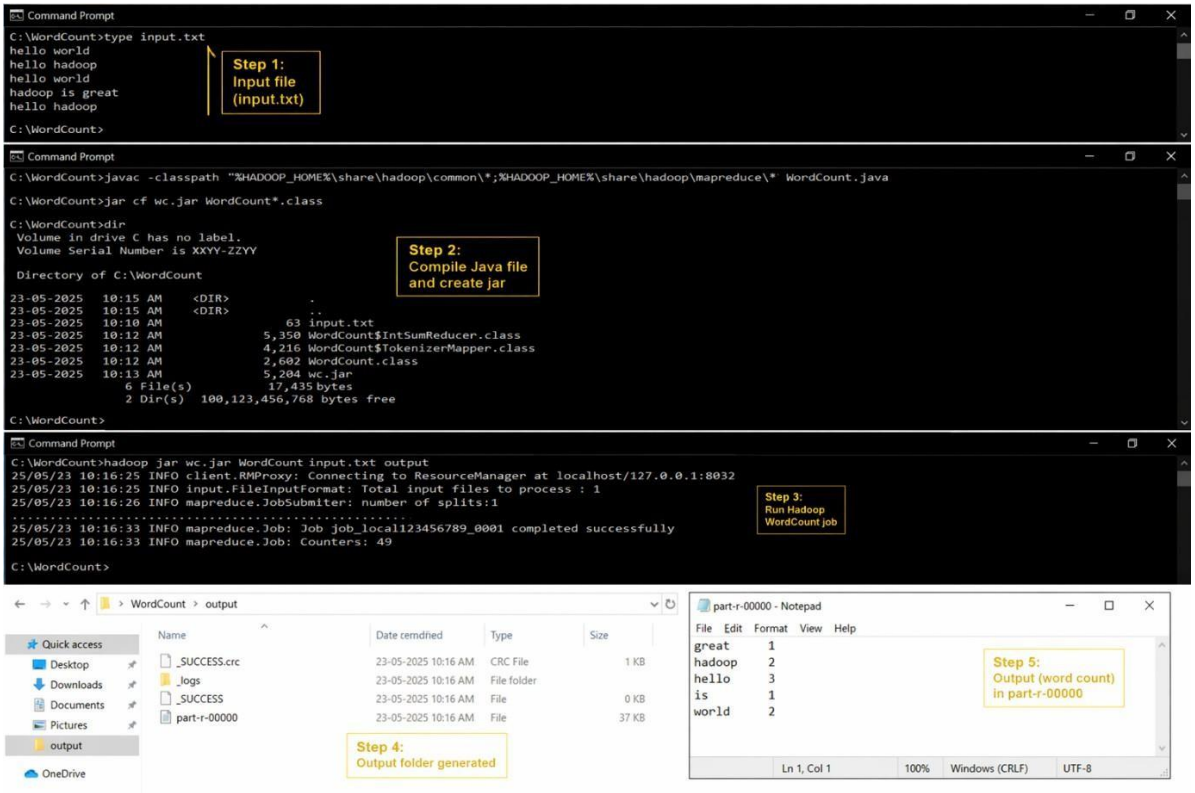
Assignment No : 11

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  // Mapper Class
  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
```

```
  public void map(Object key, Text value, Context context)
      throws IOException, InterruptedException {

    StringTokenizer itr = new StringTokenizer(value.toString());

    while (itr.hasMoreTokens()) {
      word.set(itr.nextToken());
      context.write(word, one);
    }
  }
}

// Reducer Class
public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

  private IntWritable result = new IntWritable();

  public void reduce(Text key, Iterable<IntWritable> values,
            Context context)
      throws IOException, InterruptedException {

    int sum = 0;
    for (IntWritable val : values) {
      sum += val.get();
    }

    result.set(sum);
    context.write(key, result);
  }
}
```

```
  // Driver Code (Main)
  public static void main(String[] args) throws Exception {

    if (args.length != 2) {
      System.err.println("Usage: WordCount <input path> <output path>");
      System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Word Count");

    job.setJarByClass(WordCount.class);

    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Out put:


Command Prompt
C:\WordCount>type input.txt
hello world
hello hadoop
hello world
hadoop is great
hello hadoop
C:\WordCount>
Step 1:
Input file
(input.txt)
Command Prompt
C:\WordCount>javac -classpath "%HADOOP_HOME%\share\hadoop\common*;%HADOOP_HOME%\share\hadoop\mapreduce* WordCount.java
C:\WordCount>jar cf wc.jar WordCount*.class
C:\WordCount>dir
Volume in drive C has no label.
Volume Serial Number is XXYY-ZZYY
Directory of C:\WordCount
23-05-2025 10:15 AM <DIR> .
23-05-2025 10:15 AM <DIR> ..
23-05-2025 10:10 AM 63 input.txt
23-05-2025 10:12 AM 5,350 WordCount$IntSumReducer.class
23-05-2025 10:12 AM 4,216 WordCount$TokenizerMapper.class
23-05-2025 10:12 AM 2,602 WordCount.class
23-05-2025 10:13 AM 5,204 wc.jar
6 File(s) 17,435 bytes
2 Dir(s) 100,123,456,768 bytes free
C:\WordCount>
Step 2:
Compile Java file
and create jar
Command Prompt
C:\WordCount>hadoop jar wc.jar WordCount input.txt output
25/05/23 10:16:25 INFO client.RMProxy: Connecting to ResourceManager at localhost/127.0.0.1:8032
25/05/23 10:16:25 INFO input.FileInputFormat: Total input files to process : 1
25/05/23 10:16:26 INFO mapreduce.JobSubmitter: number of splits:1
25/05/23 10:16:33 INFO mapreduce.Job: Job job_local123456789_0001 completed successfully
25/05/23 10:16:33 INFO mapreduce.Job: Counters: 49
C:\WordCount>
Step 3:
Run Hadoop
WordCount job
WordCount > output
Quick access
Desktop
Downloads
Documents
Pictures
output
OneDrive
Name
Date cemdfied
Type
Size
_SUCCESS.crc
23-05-2025 10:16 AM
CRC File
1 KB
_logs
23-05-2025 10:16 AM
File folder
_SUCCESS
23-05-2025 10:16 AM
File
0 KB
part-r-00000
23-05-2025 10:16 AM
File
37 KB
Step 4:
Output folder generated
part-r-00000 - Notepad
File Edit Format View Help
great 1
hadoop 2
hello 3
is 1
world 2
Ln 1, Col 1
100%
Windows (CRLF)
UTF-8
Step 5:
Output (word count)
in part-r-00000

Roll No: COB53

Name : Mali Akash

Batch : B3

Div : B

.....

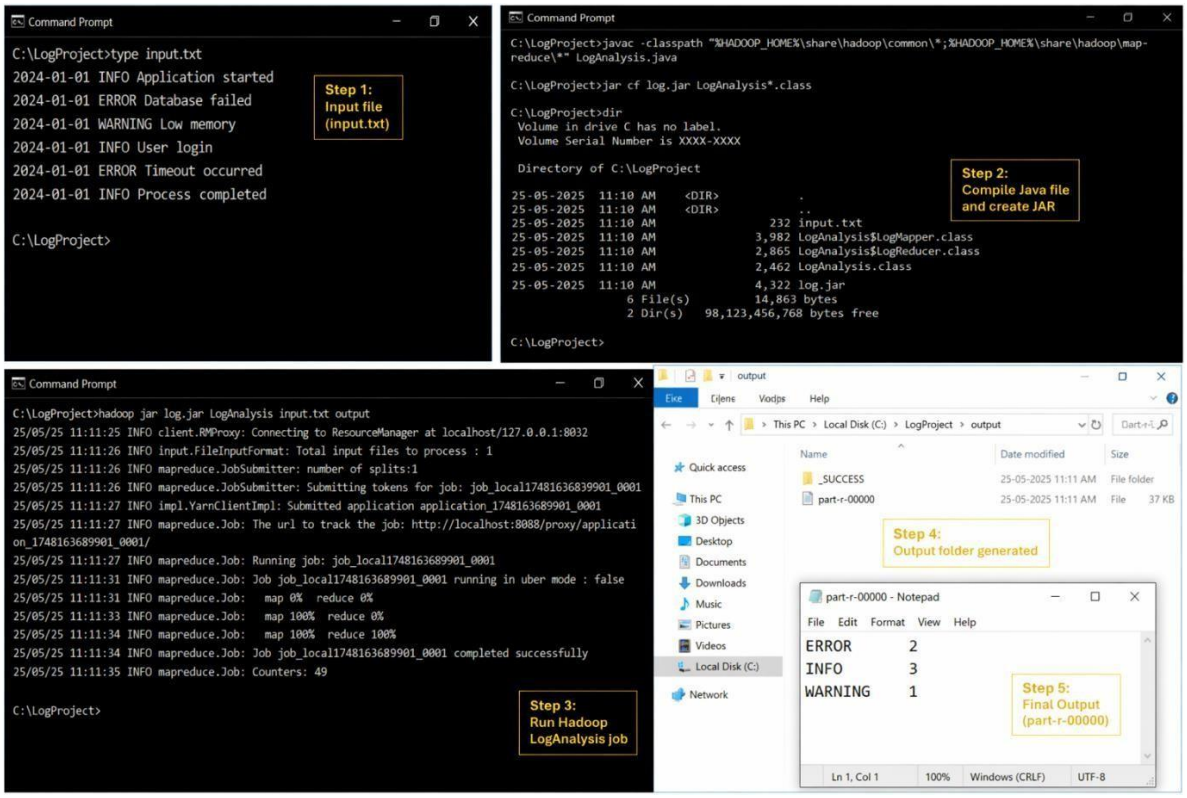
Assignment No : 12

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LogAnalysis {

  // Mapper
  public static class LogMapper
       extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);
    private Text logLevel = new Text();

    public void map(Object key, Text value, Context context)
```

```
        throws IOException, InterruptedException {

      String line = value.toString();

      if (line.contains("ERROR")) {
        logLevel.set("ERROR");
        context.write(logLevel, one);
      } else if (line.contains("WARNING")) {
        logLevel.set("WARNING");
        context.write(logLevel, one);
      } else if (line.contains("INFO")) {
        logLevel.set("INFO");
        context.write(logLevel, one);
      }
    }
}

// Reducer
public static class LogReducer
      extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context)
        throws IOException, InterruptedException {

      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
```

```
      result.set(sum);
      context.write(key, result);
    }
  }

  // Driver
  public static void main(String[] args) throws Exception {

    if (args.length != 2) {
      System.err.println("Usage: LogAnalysis <input> <output>");
      System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Log Analysis");

    job.setJarByClass(LogAnalysis.class);

    job.setMapperClass(LogMapper.class);
    job.setCombinerClass(LogReducer.class);
    job.setReducerClass(LogReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
```

}


Command Prompt
C:\LogProject>type input.txt
2024-01-01 INFO Application started
2024-01-01 ERROR Database failed
2024-01-01 WARNING Low memory
2024-01-01 INFO User login
2024-01-01 ERROR Timeout occurred
2024-01-01 INFO Process completed
C:\LogProject>
Step 1: Input file (input.txt)
Command Prompt
C:\LogProject>javac -classpath "%HADOOP_HOME%\share\hadoop\common*;%HADOOP_HOME%\share\hadoop\mapreduce*" LogAnalysis.java
C:\LogProject>jar cf log.jar LogAnalysis*.class
C:\LogProject>dir
Volume in drive C has no label.
Volume Serial Number is XXXX-XXXX
Directory of C:\LogProject
25-05-2025 11:10 AM <DIR> .
25-05-2025 11:10 AM <DIR> ..
25-05-2025 11:10 AM 232 input.txt
25-05-2025 11:10 AM 3,982 LogAnalysis$LogMapper.class
25-05-2025 11:10 AM 2,865 LogAnalysis$LogReducer.class
25-05-2025 11:10 AM 2,462 LogAnalysis.class
25-05-2025 11:10 AM 4,322 log.jar
6 File(s) 14,863 bytes
2 Dir(s) 98,123,456,768 bytes free
C:\LogProject>
Step 2: Compile Java file and create JAR
Command Prompt
C:\LogProject>hadoop jar log.jar LogAnalysis input.txt output
25/05/25 11:11:25 INFO client.RMProxy: Connecting to ResourceManager at localhost/127.0.0.1:8032
25/05/25 11:11:26 INFO input.FileInputFormat: Total input files to process : 1
25/05/25 11:11:26 INFO mapreduce.JobSubmitter: number of splits:1
25/05/25 11:11:26 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local17481636839901_0001
25/05/25 11:11:27 INFO impl.YarnClientImpl: Submitted application application_1748163689901_0001
25/05/25 11:11:27 INFO mapreduce.Job: The url to track the job: http://localhost:8088/proxy/application_1748163689901_0001/
25/05/25 11:11:27 INFO mapreduce.Job: Running job: job_local1748163689901_0001
25/05/25 11:11:31 INFO mapreduce.Job: Job job_local1748163689901_0001 running in uber mode : false
25/05/25 11:11:31 INFO mapreduce.Job: map 0% reduce 0%
25/05/25 11:11:33 INFO mapreduce.Job: map 100% reduce 0%
25/05/25 11:11:34 INFO mapreduce.Job: map 100% reduce 100%
25/05/25 11:11:34 INFO mapreduce.Job: Job job_local1748163689901_0001 completed successfully
25/05/25 11:11:35 INFO mapreduce.Job: Counters: 49
C:\LogProject>
Step 3: Run Hadoop LogAnalysis job
output
This PC > Local Disk (C:) > LogProject > output
Quick access
This PC
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
Local Disk (C:)
Network
Name
Date modified
Size
_SUCCESS
25-05-2025 11:11 AM
File folder
part-r-00000
25-05-2025 11:11 AM
File
37 KB
Step 4: Output folder generated
part-r-00000 - Notepad
File Edit Format View Help
ERROR 2
INFO 3
WARNING 1
Ln 1, Col 1
100%
Windows (CRLF)
UTF-8
Step 5: Final Output (part-r-00000)

Name : Mali Akash

Roll No: COB53

Div : B

Batch : B3

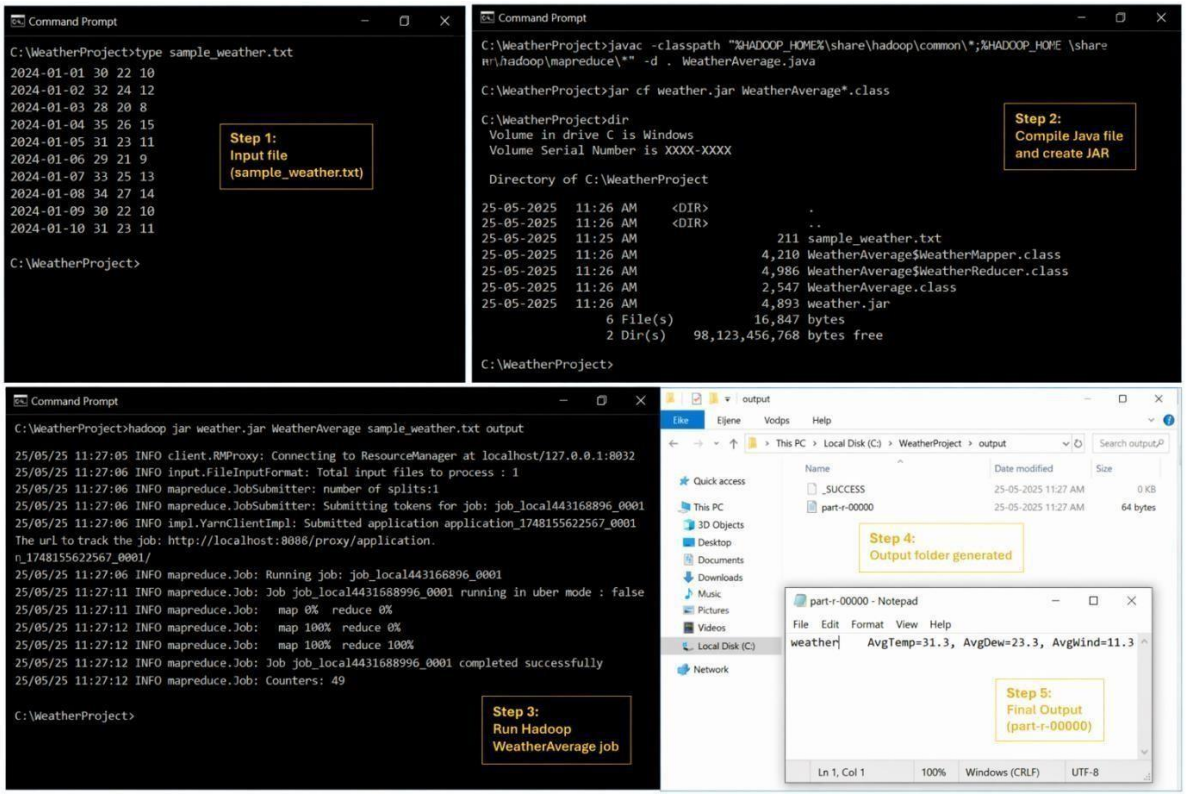
Assignment No : 13

```
import org.apache.hadoop.conf.Configuration; import
org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable; import
org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job; import
org.apache.hadoop.mapreduce.Mapper; import
org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat; import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WeatherAverage {

  // Mapper Class     public static class
WeatherMapper            extends Mapper<Object,
Text, Text, Text> {

      public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {

          String line = value.toString();
          String[] parts = line.split(" ");

          // parts[1]=Temp, parts[2]=Dew, parts[3]=Wind
if (parts.length == 4) {
```

```
            context.write(new Text("weather"),
new Text(parts[1] + "," + parts[2] + "," + parts[3]));
          }
        }
    }

    // Reducer Class     public static class
WeatherReducer            extends Reducer<Text,
Text, Text, Text> {

      public void reduce(Text key, Iterable<Text> values, Context context)
throws IOException, InterruptedException {

          int tempSum = 0, dewSum = 0, windSum = 0;
          int count = 0;

          for (Text val : values) {
             String[] nums = val.toString().split(",");

             tempSum += Integer.parseInt(nums[0]);
dewSum += Integer.parseInt(nums[1]);              windSum
+= Integer.parseInt(nums[2]);

             count++;
          }

          double avgTemp = (double) tempSum / count;
double avgDew = (double) dewSum / count;          double
avgWind = (double) windSum / count;

          String result = "AvgTemp=" + avgTemp +
```

```
                    ", AvgDew=" + avgDew +
                    ", AvgWind=" + avgWind;

            context.write(key, new Text(result));
        }
    }

    // Driver Class     public static void main(String[] args)
throws Exception {

        if (args.length != 2) {
            System.err.println("Usage: WeatherAverage <input> <output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Weather Average");

        job.setJarByClass(WeatherAverage.class);

        job.setMapperClass(WeatherMapper.class);
job.setReducerClass(WeatherReducer.class);

        job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
```

}


Command Prompt
C:\WeatherProject>type sample_weather.txt
2024-01-01 30 22 10
2024-01-02 32 24 12
2024-01-03 28 20 8
2024-01-04 35 26 15
2024-01-05 31 23 11
2024-01-06 29 21 9
2024-01-07 33 25 13
2024-01-08 34 27 14
2024-01-09 30 22 10
2024-01-10 31 23 11
C:\WeatherProject>
Step 1:
Input file
(sample_weather.txt)
Command Prompt
C:\WeatherProject>javac -classpath "%HADOOP_HOME%\share\hadoop\common*;%HADOOP_HOME \share
n\hadoop\mapreduce*" -d . WeatherAverage.java
C:\WeatherProject>jar cf weather.jar WeatherAverage*.class
C:\WeatherProject>dir
Volume in drive C is Windows
Volume Serial Number is XXXX-XXXX
Directory of C:\WeatherProject
25-05-2025 11:26 AM <DIR> .
25-05-2025 11:26 AM <DIR> ..
25-05-2025 11:25 AM 211 sample_weather.txt
25-05-2025 11:26 AM 4,210 WeatherAverage$WeatherMapper.class
25-05-2025 11:26 AM 4,986 WeatherAverage$WeatherReducer.class
25-05-2025 11:26 AM 2,547 WeatherAverage.class
25-05-2025 11:26 AM 4,893 weather.jar
6 File(s) 16,847 bytes
2 Dir(s) 98,123,456,768 bytes free
C:\WeatherProject>
Step 2:
Compile Java file
and create JAR
Command Prompt
C:\WeatherProject>hadoop jar weather.jar WeatherAverage sample_weather.txt output
25/05/25 11:27:05 INFO client.RMProxy: Connecting to ResourceManager at localhost/127.0.0.1:8032
25/05/25 11:27:06 INFO input.FileInputFormat: Total input files to process : 1
25/05/25 11:27:06 INFO mapreduce.JobSubmitter: number of splits:1
25/05/25 11:27:06 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local443168896_0001
25/05/25 11:27:06 INFO impl.YarnClientImpl: Submitted application application_1748155622567_0001
The url to track the job: http://localhost:8088/proxy/application.
n_1748155622567_0001/
25/05/25 11:27:06 INFO mapreduce.Job: Running job: job_local443168896_0001
25/05/25 11:27:11 INFO mapreduce.Job: Job job_local443168896_0001 running in uber mode : false
25/05/25 11:27:11 INFO mapreduce.Job: map 0% reduce 0%
25/05/25 11:27:12 INFO mapreduce.Job: map 100% reduce 0%
25/05/25 11:27:12 INFO mapreduce.Job: map 100% reduce 100%
25/05/25 11:27:12 INFO mapreduce.Job: Job job_local443168896_0001 completed successfully
25/05/25 11:27:12 INFO mapreduce.Job: Counters: 49
C:\WeatherProject>
Step 3:
Run Hadoop
WeatherAverage job
output
This PC > Local Disk (C:) > WeatherProject > output
Quick access
This PC
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
Local Disk (C:)
Network
Name
Date modified
Size
_SUCCESS
25-05-2025 11:27 AM
0 KB
part-r-00000
25-05-2025 11:27 AM
64 bytes
Step 4:
Output folder generated
part-r-00000 - Notepad
File Edit Format View Help
weather AvgTemp=31.3, AvgDew=23.3, AvgWind=11.3
Step 5:
Final Output
(part-r-00000)
Ln 1, Col 1
100%
Windows (CRLF)
UTF-8